Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 5:

- (Well-designed) User-defined functions
- Conditional Statements



A CSR Initiative by



NPTEL

Example

```
>>> def f(c):
...     return 1.8 * c + 32
>>> type(f)
<class 'function'>
>>> f(100)
212.0
>>> f(0)
32.0
```

parameter (comma-separated, if more than one)

parameterized expression

arguments



ven temperature **c** in Celsius to Fahrenheit

- This function is **poorly designed**:
  - Poorly named; no **purpose statement**
  - No **parameter type**, no **return type**
  - No **test cases**

NPTEL

Improvement 1: Function & parameter names

```
>>> def f(c):
...     return 1.8 * c + 32
```

- Which combination is the best **signature**?

Function name	Parameter name
convert	C
C_to_F	temp
to_fahrenheit	celsius

```
1   def to_fahrenheit(celsius):
2       return celsius * 9/5 + 32
```

NPTEL

Improvement 2: Purpose statement

- In Python: **docstring**
- It explains ***why*** the function was written
  - Not ***how*** it works

```
1   def to_fahrenheit(celsius):
2       """Convert a given temperature in
3       Celsius to Fahrenheit, as long
4       as it is above absolute zero."""
5       if celsius < -273.15:
            return "Temperature is below absolute zero"
        else:
            return celsius * 9/5 + 32
```

If not?

NPTEL

Critique Our Friend's Function

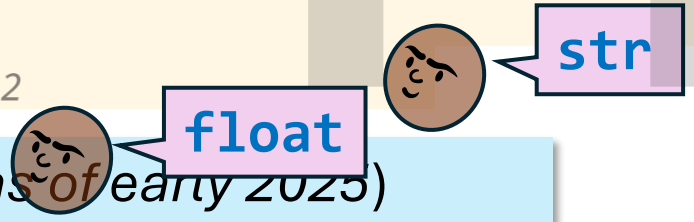
```
1    def next(n):
2        """Return the next even
3        number after n by adding
4        1 and then another 1 if
5        n is odd. If n is even,
6        just add the first 1."""
7        return n + 1 + (n % 2)
```

NPTEL

Improvement 2: Parameter & return types

- In Python: **type-hinting**
  - Only “hinting” (Python will not check this)

```
1   def to_fahrenheit(celsius: float) -> float:
2       """Convert a given temperature in
3       Celsius to Fahrenheit, as long
4       as it is above absolute zero."""
5       if celsius < -273.15:
            raise ValueError("Temperature is below absolute zero.")
        return celsius * 9/5 + 32
            return celsius * 9/5 + 32
```



Important Observation (*as of early 2025*)
Gen AI tools suggest missing details **silently**.
These suggestions are often (but not ***always***) helpful.

Improvement 3: Test cases

- In Python: **doctests**
  - Pretend to use the REPL in the docstring

```
1  def to_fahrenheit(celsius: float) -> float:
2      """Convert a given temperature in
3      Celsius to Fahrenheit, as long
4      as it is above absolute zero.
5      >>> to_fahrenheit(-300.0)
6      nan
7      >>> to_fahrenheit(-273.15)
8      0.0
9      """
```

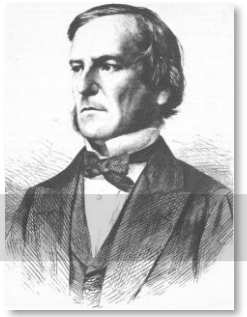
Not a Number

0 - 273.15 × 1.8 + 32 =

-459.67

Do not trust Gen AI suggestions blindly!

Simple Conditional Statements



George Boole

Source: Wikipedia
Two interesting connections with India!

- Simplest form:

```
if condition:
    do_something
```

- ***condition*** is usually a **Boolean** expression
  - Relational operators: <, >, <=, >=, ==, !=
    - For `str`: lexicographic order
  - Object identity: `is`, `is not`
  - Membership: `in`, `not in`
  - Boolean operators: `and`, `or`, `not`
- But ***condition*** can be any object/expression

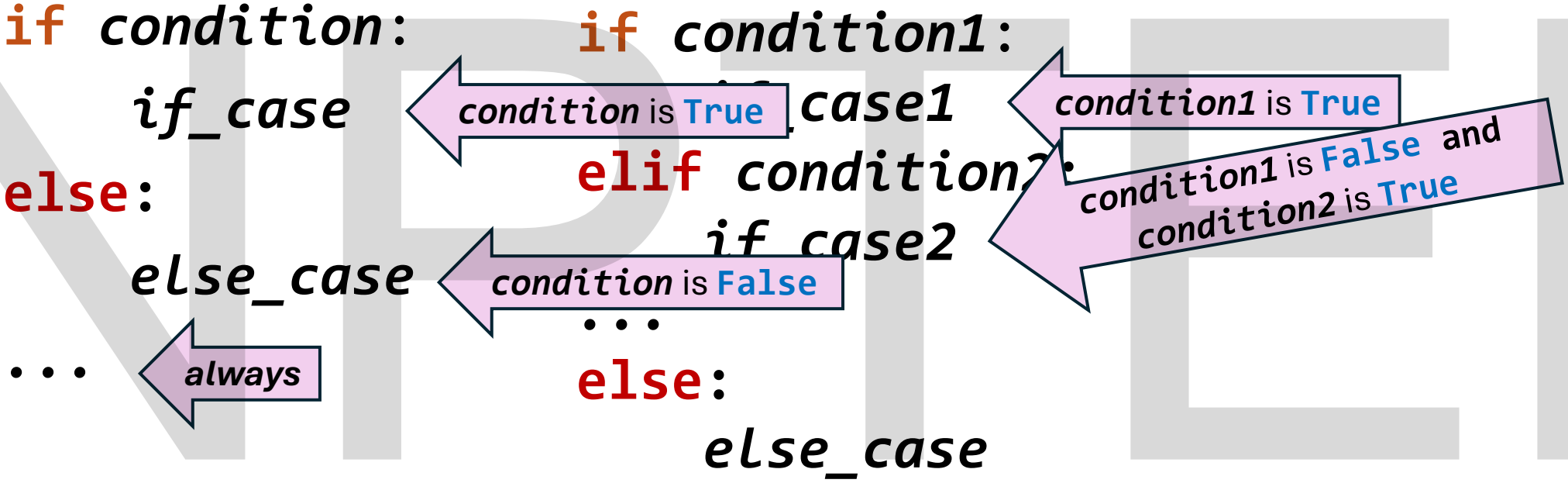
NPTEL

Critique this AI-generated Suggestion

```python
def is_multiple(n: int, m: int) -> bool:
    """Check if n is an integer multiple
    of m.
    >>> is_multiple(10, 5)
    True
    >>> is_multiple(10, 0)
    False
    """
    if m == 0:
        return False
    return n % m == 0
```

NPTEL

Complex Conditional Statements



- Programs can have complex **control flow** and **data flow**
- Visualize using PythonTutor

Critique this Function

```
1   def median(a: int, b: int, c: int) -> int:
2       """Calculate the median of 3 ints.
3       >>> median(1, 2, 3)
4       2
5       >>> median(9, 3, 6)
6       6
7       >>> median(7, 8, 0)
8       7
9       """
10      if a <= b <= c:
11          return b
12      elif b <= a <= c:
13          return a
14      else:
15          return c
```

```
a <= b and b <= c
```

NPTEL

Summary

- A well-defined Python function has:
  - A meaningful **signature** (function and **parameter** names) and **type-hinting**
  - A **docstring** explaining ***what*** the function does (i.e., its purpose)
  - One or more **doctests**, especially for **corner-case** inputs
- Conditional statements help us write programs with complex **control flow**
- Don’t forget to **critique** AI-generated code!

NPTEL